

Tidal2WiiM

Code Atlas

Explore the code. Follow the data flow. Plan changes.

19 module profiles · 46 classes and interfaces · 14 PlantUML models

This package explains the existing app, from startup through local storage and TIDAL integration to the artist editor.

The PlantUML models were created with AI assistance (ChatGPT) according to specific instructions.

The existing ER and table models remain unchanged. The new class and sequence models complement them with a view of the program code.

Contents

Source revision and orientation	3
Quick overview	4
Architecture and reading guide	6
Module handbook	8
Data exchange and storage	18
Model 01 · Responsibilities and data flows	20
Model 02 · Domain models in memory	21
Model 03 · Session management and testability	22
Model 04 · Artist editor: input, draft and storage	23
Model 05 · User interface: objects and callbacks	24
Model 06 · App startup: cache and stored session	25
Model 07 · Interactive sign-in and secure storage	26
Model 08 · Loading the library: foreground and background	27
Model 09 · From JSON resources to Album objects	28
Model 10 · API 401: refresh once and retry	29
Model 11 · Editing and saving sort names in bulk	30
Model 12 · Assigning an album to several categories	31
Model 13 · Cover display: file, download and fallback	32
Model 14 · Loading album details and handing off to TIDAL	33
External packages and tests	34
Complete class index	36
Working with the sources	38

The chapters and models can be reached through the table of contents and the PDF bookmarks. The 19 modules have fixed identifiers from M01 to M19. Each profile identifies the model that explains its flow.

Reading pages: A4. Model sheets: A3. Editable PlantUML sources, SVG and PNG files are provided in the ZIP archive.

Source revision and orientation

The documentation describes a fixed source revision. The source links in the module profiles lead to this exact public commit.

Source code: `Code40vM/tidal2wiim-demo`
`851304ffd33e6ed9f65d74a29d146311e9a6265b`

What does "module" mean here?

Each profile describes one of the project's own Dart files under `lib/`. Technically, the project is a single Dart package with three libraries of its own. The folders group responsibilities, but they are not separately installable packages.

Start with the artist editor

A simple reading route leads through M05 (draft), M16 (editor page), M07 (saving) and Model 11. It shows why input initially stays in RAM and only reaches SQLite after an explicit save action.

Tools and files

The app is built with Flutter and Dart, while SQLite stores the user's personal organisation locally. VS Code is the development environment; all modelling is done in PlantUML. GitHub keeps source revisions and documentation together in a traceable way. The specific runtime and development dependencies are listed in the chapter "External packages and tests".

Quick overview

The app is a digital record shelf for TIDAL albums. It adds personal organisation by artist and category. It processes metadata and cover images; the TIDAL app then plays the music.

Each module in one sentence

The identifiers M01 to M19 are consistent across the handbook, the individual profiles and the PDF. Here, a module means a project-specific Dart file. The entire project is a Dart package; the folders grouped by responsibility are not individually installable packages.

Module / file	Responsibility
M01 - lib/main.dart	Starts Flutter and sets the app theme; opens HomePage.
M02 - lib/core/hilfen.dart	Groups helper functions for text, names, search and JSON.
M03 - lib/core/sammlung.dart	Filters and sorts albums; creates artist folders and the A-Z index.
M04 - lib/models/modelle.dart	Defines the shared data objects, such as albums, artists and categories.
M05 - lib/models/kuenstler_sortierung.dart	Keeps unsaved sort names as an editable draft.
M06 - lib/data/lokale_datenbank.dart	Stores and reads metadata, categories and artist preferences in SQLite.
M07 - lib/data/kuenstler_sortierung_speichern.dart	Writes several changed artist sort names in one transaction.
M08 - lib/services/tidal_auth_service.dart	Restores the session, refreshes tokens and encapsulates the secure store.
M09 - lib/services/tidal_api.dart	Reads TIDAL JSON; handles status, cursors, cover URLs and error messages.
M10 - lib/services/cover_cache.dart	Downloads cover images and keeps them as local image files.
M11 - lib/pages/home_page.dart	Coordinates navigation, sign-in, collection and background synchronisation.
M12 - lib/pages/start_page.dart	Displays the start image and status; passes user actions on through callbacks.
M13 - lib/pages/album_detail_page.dart	Displays album details, loads tracks and manages category assignments.
M14 - lib/pages/kuenstler_page.dart	Displays the already prepared albums in an artist folder.
M15 - lib/pages/kategorie_page.dart	Matches stored category IDs to the currently loaded albums.
M16 - lib/pages/kuenstler_sortiernamen_page.dart	Displays the artist editor with search, input fields and saving.
M17 - lib/widgets/album_grid.dart	Displays a reusable album grid and opens album details.
M18 - lib/widgets/album_cover.dart	Displays a cover image from the file cache or a fallback display.
M19 - lib/widgets/dialoge.dart	Collects dialog input and returns results to the calling page.

Data follows different paths

HomePage coordinates the processes. Pages receive objects and services through constructors. They return user actions or changes through callbacks and Navigator results. Asynchronous results are handled with Future and await.

Data	Path / location	Why separate?
Album metadata	TIDAL JSON → album objects → SQLite cache	Previously loaded albums are quickly available at the next start.
Personal organisation	Dialog/editor → result objects → SQLite	Categories and sort names are retained locally.
Cover images	HTTPS image request → files → AlbumCover	Image bytes belong in the file cache, not in SQL tables.
Sign-in	Authentication service ↔ secure session store	Tokens are stored separately from metadata and images.
Playback	Album/track link → external handler/TIDAL app	The app passes on a content ID; it does not stream music itself.

Three useful starting points for the models

1. Component model 01: an overview of responsibilities, data flows and system boundaries.
2. Class model 04: how the editor, draft, change list and storage relate to each other.
3. Sequence model 11: enter values → build a change list → save together → reload the view.

Next, model 06 covers app startup, model 08 covers full library synchronisation and model 13 covers cover images. The handbook links each module to the relevant diagrams.

Architecture and reading guide

A short path through the project

1. `main.dart`: identify the entry point and library boundaries.
2. `models/modelle.dart`: understand which data the app uses in RAM.
3. `core/sammlung.dart`: derive artist folders and views from albums.
4. `pages/home_page.dart`: follow the coordination of the UI, cache and online synchronisation.
5. `data/` and `services/`: distinguish persistent data, sign-in and external access.
6. Follow the artist editor with diagram 11: initial state → draft → confirmed change list → transaction → reloaded view.

Three project-specific Dart libraries

Library	Connection	Meaning
<code>main.dart</code> plus 16 part files	part / part of	Shared namespace. <code>_private</code> names are visible across files within this library.
<code>models/kuenstler_sortierung.dart</code>	import in <code>main.dart</code>	Pure editing model without Flutter, SQLite or TIDAL dependencies.
<code>services/tidal_auth_service.dart</code>	import in <code>main.dart</code>	Encapsulated session with public methods and private internal state.

The folders grouped by responsibility are therefore not strict technical layers. `HomePage` is the central stateful UI component (`StatefulWidget`) of the Flutter app. Its state, `_HomePageState`, coordinates navigation, views and data retrieval. The start view with its title graphic is built by the `StartPage` widget. The database and API use the same domain models. Refactoring into further independent libraries would be a later change, rather than the current implementation.

How the connections work

Constructor parameters: When a page is opened, the album, lists, optional country and existing `AuthService` reference are passed to it. The data is not exchanged again over a network between the Dart files.

Future and await: A `Future` represents a result that will become available later. `await` waits within the relevant asynchronous function. `Future.wait` does not start separate operating system threads; the asynchronous operations that have been started can overlap in time. The cover workers are also asynchronous loops, not explicitly created isolates.

Callbacks: A child page calls a function passed to it. `onKategorienGeaendert()` does not send a complete data set. The parent view subsequently reads the affected data again. In contrast, the editor explicitly passes its change list to `onSpeichern(aenderungen)`.

UI state: `setState` notifies Flutter of a local state change. It does not save anything in SQLite. `ChangeNotifier` is used specifically for the session service; there is no general event bus here.

Navigator: A dialog or route can return a result. On editor success, this is true; only then does `HomePage` reload the settings. Cancelling or going back without changes does not result in a confirmed save operation.

Notation in the PlantUML models

Solid class arrows show held references; dashed arrows show usage or creation. A dashed relationship labelled “via IDs” is not a stored object reference. Realisation and inheritance are shown only where they exist in the Dart code. The database extension is not a subclass.

In sequence diagrams, time runs from top to bottom. `alt` shows alternatives, `opt` a condition, `loop` repetition and `par` independently started asynchronous operations. Return arrows carry data or signal the completion of a Future. The representation condenses the code; it is not a runtime trace.

The class diagrams show selected attributes and methods for readability. The complete directory of all 46 classes/interfaces is provided separately. Helper functions are not drawn as invented service classes.

Module handbook

For each file: responsibility, inputs, outputs, connections and a note. Full symbol names are retained so that they can be searched for directly in the code.

M01 · Start the app and connect the files

lib/main.dart

Responsibility: Starts Flutter with Tidal2WiiMApp. Builds MaterialApp, a dark theme and HomePage. Combines 16 part files into one shared Dart library and imports two other project libraries.

Input: Optional: bibliothekLaden callback and TidalAuthService for isolated tests.

Output: Widget tree with HomePage; does not itself provide album data or run SQL queries.

Connections: Includes all part files; imports models/kuenstler_sortierung.dart and services/tidal_auth_service.dart.

In the code: main(), Tidal2WiiMApp.build()

Note: The folders represent functional areas. Technically, tidal2wiim is a Dart package with three project libraries.

Related models: 01, 05, 06

M02 · Process JSON, search text and durations

lib/core/hilfen.dart

Responsibility: Provides small helper functions with no state of their own. Checks JSON structures, cleans up text and converts **time spans in the standardised format defined by ISO 8601** - here, for example, the playing time of a music track - for the track display.

Input: Object?, JSON values, search text or Duration?.

Output: Json, List<Json>, String?, normalised search text, as well as Duration? and display text.

Connections: Used by models, API response processing, the collection and the album detail page. No direct network or database access.

In the code: Json, _objekt(), _optionalObjekt(), _liste(), _text(), _vergleichstext(), _dauerLesen(), _dauerAnzeige()

Note: A missing optional value may remain null. An invalid required structure, however, is reported as a FormatException.

Data types: String? means text or null. Duration? means a duration or null. In each case, the ? allows a missing value.

Related models: 09, 14

M03 · Derive views from existing albums

lib/core/sammlung.dart

Responsibility: Filters and sorts albums, creates artist folders and calculates the A-Z jump index. Defines the selection values for the app page, collection view and sorting.

Input: List<Album>, search text, Sortierung and Map<String, KuenstlerEinstellung>.

Output: Filtered album list, List<KuenstlerOrdner> or Map<String, int> as a letter index.

Connections: HomePage calls the functions when building the collection. Uses domain models and text helpers.

In the code: Sortierung, SammlungAnsicht, AppSeite, _ansichtErstellen(), _kuenstlerOrdnerErstellen(), _kuenstlerAlphabetIndex()

Note: Artist folders are created in RAM. Multiple artists on one album result in multiple folder assignments. Album sorting uses TIDAL artist names; custom sort names control the artist folders.

Related models: 01, 02

M04 · Describe shared data objects

lib/models/modelle.dart

Responsibility: Defines eight domain and data transfer models for albums, artists, categories, settings, tracks, folders, API pages and the local cache. Album.ausRessourcen() combines API resources into an album.

Input: Constructor values; resource index keyed by type:id; SQLite map for KuenstlerEinstellung.

Output: Typed Dart objects; derived values such as year, display name and album count.

Connections: Shared vocabulary for HomePage, detail pages, SQLite, the collection and cover display.

In the code: Album, Kuenstler, Kategorie, KuenstlerEinstellung, KuenstlerOrdner, AlbumTitel, AlbumSeite, LokaleBibliothekCache

Note: Kategorie holds album IDs. Album contains artists, but no track list. A final field does not automatically make a list referenced by that field immutable.

Related models: 02, 09

M05 · Collect changes in a draft

`lib/models/kuenstler_sortierung.dart`

Responsibility: Manages the original rows and sort names that have not yet been saved. Detects actual changes and keeps the order and the basis for searching stable while typing.

Input: `Iterable<KuenstlerSortierZeile>`; artist ID and input text; search text.

Output: Filtered rows, change count and an immutable `List<KuenstlerSortierAenderung>`.

Connections: Independent library with no imports from Flutter, SQLite or TIDAL. The editor page uses it; dedicated unit tests check the logic.

In the code: `KuenstlerSortierZeile`, `KuenstlerSortierAenderung`, `KuenstlerSortierEntwurf`; `setzen()`, `filtern()`, `aenderungen`

Note: Raw text is retained while typing. Only the effective value to be saved is trimmed; an empty field falls back to the original TIDAL name.

Related models: 04, 11

M06 · Store structured data locally

`lib/data/lokale_datenbank.dart`

Responsibility: Opens `tidal2wiim.db` with schema 3 and enables foreign keys. Encapsulates the cache, artist preferences, categories and album assignments; handles migrations and transactions.

Input: Album lists with timestamps, `KuenstlerEinstellung`, category names, and album and category IDs.

Output: `Future<LokaleBibliothekCache?>`, settings map, category lists, sets of IDs or `Future<void>`.

Connections: `HomePage`, `KategoriePage` and `AlbumDetailPage` use `LokaleDatenbank.instance`. `sqlite` executes SQL; `path` constructs the file path.

In the code: `LokaleDatenbank`, `albumCacheLaden()`, `albumCacheSpeichern()`, `kategorienLaden()`, `albumKategorienSetzen()`

Note: Cache replacement and assignment changes are atomic. Only `category_albums.category_id` is an SQL foreign key; artist data is stored in the cache as JSON.

Related models: 01, 08, 12

M07 · Write sort names to SQLite in one batch

`lib/data/kuenstler_sortierung_speichern.dart`

Responsibility: Adds batch saving to LokaleDatenbank. Checks IDs in advance and performs all changes in a single transaction. Does not modify existing custom display names.

Input: List<KuenstlerSortierAenderung> from the draft.

Output: Future<void>; on errors, an Exception and rollback rather than a partially applied set of changes.

Connections: HomePage connects the extension method to the editor page as the onSpeichern callback. Writes to artist_preferences.

In the code: extension KuenstlerSortierungSpeichern on LokaleDatenbank;
kuenstlerSortiernamenSpeichern()

Note: An extension is not a subclass. When resetting, a preference record is only deleted if no custom display name needs to be retained.

Related models: 04, 11

M08 · Manage login and the token lifecycle

`lib/services/tidal_auth_service.dart`

Responsibility: Restores the shared session, saves login results and renews tokens when needed. Prevents duplicate concurrent refreshes and results from a session that has since been replaced.

Input: clientId, optional store/client/clock dependencies; TidalSession at login; HTTPS URI and HTTP client when reading.

Output: Future<String> for a token, Future<Response> for an API GET, status getters and ChangeNotifier notifications.

Connections: HomePage owns or receives the service and passes the same reference to subpages. SecureTidalSessionStore uses flutter_secure_storage.

In the code: TidalAuthService, TidalSession, TidalSessionStore, SecureTidalSessionStore; restore(), acceptLogin(), accessToken(), get(), forget()

Note: HTTP 401 leads to at most one second API GET. Temporary network errors preserve the session. storageWarning indicates problems reading, saving or removing the securely stored session.

Related models: 03, 06, 07, 10

M09 · Standardise API calls and read JSON

lib/services/tidal_api.dart

Responsibility: Builds OpenAPI URLs, calls the authentication service, checks for HTTP 200 and decodes JSON. Reads the next-page cursor, selects a suitable cover URL and translates errors into understandable messages.

Input: http.Client, shared TidalAuthService, path segments and query parameters; JSON resources for the cover and cursor.

Output: Future<Json>, String? for a cursor/cover URL, or ApiFehler.

Connections: HomePage and AlbumDetailPage use _apiLesen(). Album.ausRessourcen() uses _coverUrlLesen().

In the code: _apiLesen(), _cursorLesen(), _coverUrlLesen(), _fehlerText(), ApiFehler

Note: The file is mainly a collection of functions, not a TidalApi class. Each call waits 300 ms before the request; this is not a complete automatic retry mechanism for HTTP 429.

Related models: 01, 09, 10

M10 · Cache album covers as files

lib/services/cover_cache.dart

Responsibility: Finds or downloads cover files in the local cover_cache directory. Combines ongoing downloads for the same target path, initially writes to .tmp and removes obsolete files during synchronisation.

Input: Album with an ID and cover URL, or List<Album> for preloading; optional HTTP client.

Output: Future<File?> for a cover, or Future<void> after synchronisieren().

Connections: AlbumCover requests individual files. HomePage starts preloading after accepting the cache or completing a full update.

In the code: LokalerCoverCache.instance, dateiFuer(), synchronisieren(), _herunterladen()

Note: The limit of up to four workers applies to synchronisieren(), rather than being a global limit on all individual requests. Image bytes are stored in the file system and never as BLOBs in SQLite.

Related models: 01, 06, 13

M11 · Coordinate app state and workflows

[lib/pages/home_page.dart](#)

Responsibility: The central StatefulWidget HomePage and its state _HomePageState manage the start view, lab and collection. They coordinate login, the cache, API pages, background synchronisation, search, artist folders, categories and navigation to the sort-name editor.

Input: Optional test dependencies, user actions, service notifications, API and database results.

Output: Current UI state; data and AuthService passed to child pages; callbacks for reloading.

Connections: The central coordinator for the database, authentication, API and cover cache. Builds StartPage; opens artist, category, album and editor pages.

In the code: HomePage, _HomePageState; _initialisieren(), _seiteLesen(), _albenLaden(), _bibliothekImHintergrundAktualisieren(), _kuenstlerSortiernamenOeffnen()

Note: Splitting the code into files does not yet separate all UI and domain logic. The lab and collection are widget methods here, rather than separate page classes.

Related models: 01, 05, 06, 08, 09, 11

M12 · Connect the start image to real controls

[lib/pages/start_page.dart](#)

Responsibility: Displays the start graphic, dynamic album count and status bar. Scales the image, text and touch target areas together and triggers four actions supplied by the parent widget.

Input: Album count, session/loading flags, update time and four callbacks.

Output: Calls to onSammlungOeffnen, onLaborOeffnen, onAktualisieren and onTidalStatus.

Connections: Configured by HomePage. _StartAktion, _StartStatusLeiste and two CustomPainter implementations form internal UI components.

In the code: StartPage, _StartAktion, _StartStatusLeiste, _StartTidalZeichen, _StartBibliothekZeichen

Note: StartPage does not read SQLite or the API itself. It displays the supplied state and delegates actions back to HomePage.

Related models: 05, 06

M13 · Display an album, its tracks and local assignments

`lib/pages/album_detail_page.dart`

Responsibility: Loads the track list from TIDAL and categories from SQLite independently of each other. Allows category assignment and opens album, track or video links externally.

Input: Album, shared AuthService, optional country and callbacks for login and category changes.

Output: List<AlbumTitel> in the page state, local category selection and external URLs; feedback after an assignment is saved.

Connections: Opened by HomePage or AlbumGrid; uses `_apiLesen()`, `LokaleDatenbank`, `AlbumCover` and dialogs.

In the code: `AlbumDetailPage`, `_AlbumDetailPageState`; `_titelLaden()`, `_kategorienBearbeiten()`, `_inTidalOeffnen()`, `_titelInTidalOeffnen()`

Note: The track list is not cached persistently. `launchUrl()` starts an external handler; playback and WiiM selection then take place outside our app.

Related models: 05, 12, 14

M14 · Display the albums in an artist folder

`lib/pages/kuenstler_page.dart`

Responsibility: A small, stateless wrapper: displays the display name as the page title and passes the associated albums to AlbumGrid.

Input: KuenstlerOrdner, AuthService, optional country and change/login callbacks.

Output: Widget tree consisting of AppBar and AlbumGrid.

Connections: HomePage creates the folder beforehand with `_kuenstlerOrdnerErstellen()`. AlbumGrid handles navigation to the details.

In the code: `KuenstlerPage.build()`

Note: The page does not create folders or request artist data. It receives objects that have already been prepared.

Related models: 02, 05

M15 · Connect saved assignments to loaded albums

lib/pages/kategorie_page.dart

Responsibility: Reads the category together with its album IDs and filters the supplied album list. Displays both the visible albums and the number of saved assignments.

Input: Category ID/name, all currently loaded albums, AuthService and callbacks.

Output: List<Album> sorted by year and title for AlbumGrid; updates after category changes.

Connections: Uses LokaleDatenbank.kategorienLaden(); AlbumGrid opens the details. The callback reloads this page first and then the parent view.

In the code: KategoriePage, _KategoriePageState; _neuLaden()

Note: A saved album ID may no longer be in the current cache. The total count and visible count can therefore differ.

Related models: 05, 12

M16 · Edit multiple sort names conveniently

lib/pages/kuenstler_sortiernamen_page.dart

Responsibility: Provides search, input fields, a change counter and saving. Manages controllers, focus changes and protection against leaving with unsaved changes.

Input: List<KuenstlerSortierZeile> and an onSpeichern callback.

Output: A list of changes passed to the callback; a Navigator result of true after successful saving. Cancelling does not return true.

Connections: HomePage supplies the rows and save function. KuenstlerSortierEntwurf keeps the domain logic separate from Flutter.

In the code: KuenstlerSortiernamenPage, _KuenstlerSortiernamenPageState; _speichern(), _verlassen(), _fokusVerschieben()

Note: If saving fails, the entered values remain on the page. Editing and accidental closing are blocked while saving.

Related models: 04, 11

M17 · A reusable grid for child pages

`lib/widgets/album_grid.dart`

Responsibility: Displays album cards with a cover, title, artists and year. Tapping a card opens an AlbumDetailPage with the same services and callbacks.

Input: List<Album>, AuthService, optional country and two callbacks.

Output: Album grid and navigation to the detail page.

Connections: Used by KuenstlerPage and KategoriePage; builds AlbumCover. The main collection currently has its own grid in HomePage.

In the code: AlbumGrid.build()

Note: Reuse is clearly visible here. It would be inaccurate to claim that every grid in the app already uses this widget.

Related models: 05, 14

M18 · Load a cover file and handle image errors

`lib/widgets/album_cover.dart`

Responsibility: Requests a cover from the file cache and displays a loading indicator, a file image or a network image. A placeholder appears if the URL is missing or the network image fails.

Input: Album; changes to the ID or cover URL are detected when the widget is updated.

Output: Image display with FutureBuilder; no modified Album object.

Connections: Uses LokalerCoverCache.dateiFuer(), Image.file and Image.network as a fallback. Used in the main collection, AlbumGrid and the detail page.

In the code: AlbumCover, _AlbumCoverState; initState(), didUpdateWidget(), _netzFallback()

Note: An existing file path does not guarantee that the image can be decoded. Image.file therefore also has error handling.

Related models: 13

M19 · Collect input and return results

lib/widgets/dialoge.dart

Responsibility: Groups three stateful dialogs: a category name, an individual artist adjustment and album category selection. Each dialog has its own input controllers or copy of the selection.

Input: Dialog title/initial name, artist data or categories with selected IDs.

Output: String, _KuenstlerDialogErgebnis or Set<int> through Navigator.pop(); null when cancelled.

Connections: HomePage and AlbumDetailPage open the dialogs and then save to SQLite themselves.

In the code: _KategorieNameDialog, _KuenstlerBearbeitenDialog, _AlbumKategorienDialog, _KuenstlerDialogErgebnis; corresponding State classes

Note: The dialogs do not perform SQL operations. Controllers are released in dispose() of their own dialog state.

Related models: 12

Data exchange and storage

Data contracts between modules

Connection	Data type / transfer	Meaning
API → _apiLesen()	HTTP response / UTF-8 JSON	HTTP 200 is checked; the result is <code>Json = Map<String, dynamic></code> .
JSON → Album	<code>Map<String, Json></code> keyed by <code>type:id</code>	Collection IDs are resolved using album, artist and cover resources.
Page loader → Home	<code>AlbumSeite</code>	Contains <code>List<Album></code> and the next collection cursor, rather than the complete library.
SQLite → Home	<code>LokaleBibliothekCache?</code>	Album list and storage timestamp; an empty table returns null.
Home → child page	Objects and references	Album/folder/lists plus the same <code>TidalAuthService</code> instance; no copied token.
Dialog → caller	<code>String</code> / <code>Set<int></code> / result object	<code>Navigator.pop</code> provides a result later; null usually means cancellation.
Editor → save function	<code>List<KuenstlerSortierAenderung></code>	Only artists that have actually changed; after success, <code>pop(true)</code> returns to Home.
Detail page → parent view	<code>Future<void> Function()</code>	<code>onKategorienGeaendert</code> reports the change without a data-set payload; parents read again.
Authentication → Home	<code>ChangeNotifier</code>	The listener reads the status getters again and calls <code>setState</code> ; no app-wide event bus.
Cover cache → widget	<code>Future<File?></code>	Image bytes remain in the file system. null/file errors lead to the network fallback.
App → TIDAL app	HTTPS URL	Only the content ID is in the link; neither an OAuth token nor a music file is passed on.

Four forms of storage

Location	Content	Lifetime / boundary
RAM	Albums, filtered views, loaded tracks, editor draft	Lost when the process ends; <code>setState</code> only updates the UI.
SQLite / tidal2wiim.db	Categories, album assignments, artist preferences, metadata cache	Persistent within the app's storage area. Schema 3; no audio streams or tokens.
Files / cover_cache	Downloaded covers as .img; temporarily .tmp	File name based on a sanitised album ID and a stable URL hash; can be downloaded again later.
Secure store	Version, clientId, access/refresh tokens and expiry time as JSON	Encapsulated by <code>TidalSessionStore</code> ; not in SQLite or cover files.

SQLite is not the class model

artist_preferences stores custom artist names and sort names. categories contains categories. category_albums maps category/album pairs. album_cache contains album metadata, order and artist data as JSON.

Only category_albums.category_id → categories.id is an SQL foreign key. There is no such foreign key to album_cache. A complete cache replacement therefore does not delete category assignments. artist_preferences also only refers logically to TIDAL artist IDs; there is no separate artist table in the current schema.

A KuenstlerOrdner is derived from the current album objects. AlbumSeite and LokaleBibliothekCache are transport objects. AlbumTitel is loaded when a detail page is opened and is not stored in the local database. The existing ER/table models remain separate, unchanged representations.

Example: a new sort name

Editor rows are created from album data and stored settings. The input “Wülker, Nils” initially becomes text in the draft. Saving creates a KuenstlerSortierAenderung object with the artist ID, TIDAL name and new sort name. The extension updates artist_preferences within a transaction. HomePage then reads the settings again and recalculates the artist folders and A-Z index. The TIDAL name and any existing custom display name are preserved.

Example: an updated album

The collection first supplies the album ID. Further API calls supply album, artist and cover resources. Album.ausRessourcen() creates the domain object. After full background synchronisation, the entire metadata cache is replaced atomically. Only then is the visible album list replaced. The cover image is downloaded separately as a file. A single failed cover download does not invalidate the successfully stored album record.

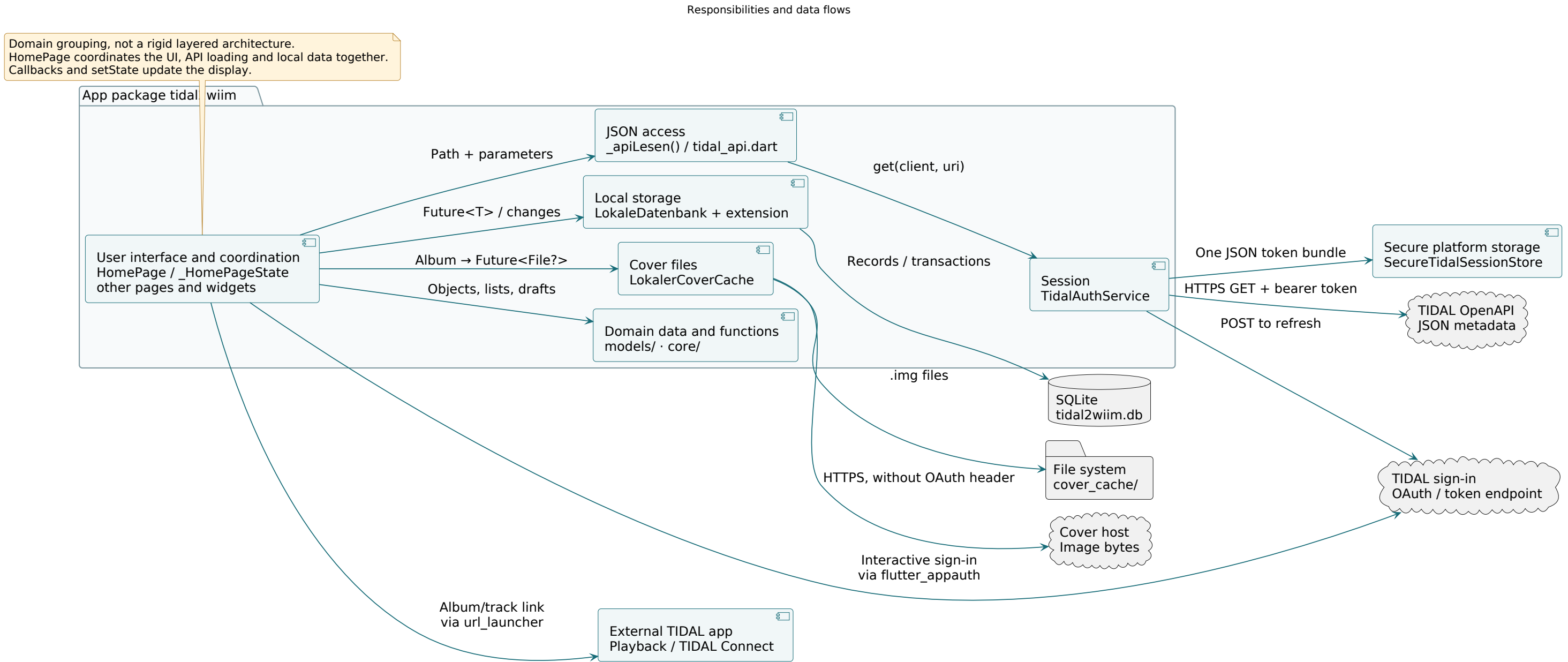
System boundary for playback

Album and track links contain a content ID. They contain no session tokens. The platform chooses the external handler; on the intended tablet, this is the TIDAL app. Playback must be started there. TIDAL Connect to the WiiM is not a separate call made by the code examined here.

Model 01 · Responsibilities and data flows

The app processes metadata and personal organisation. The music stream and WiiM control are outside its Dart code.

Source locations under lib/, unless stated otherwise: main.dart; pages/home_page.dart; services/*; data/*



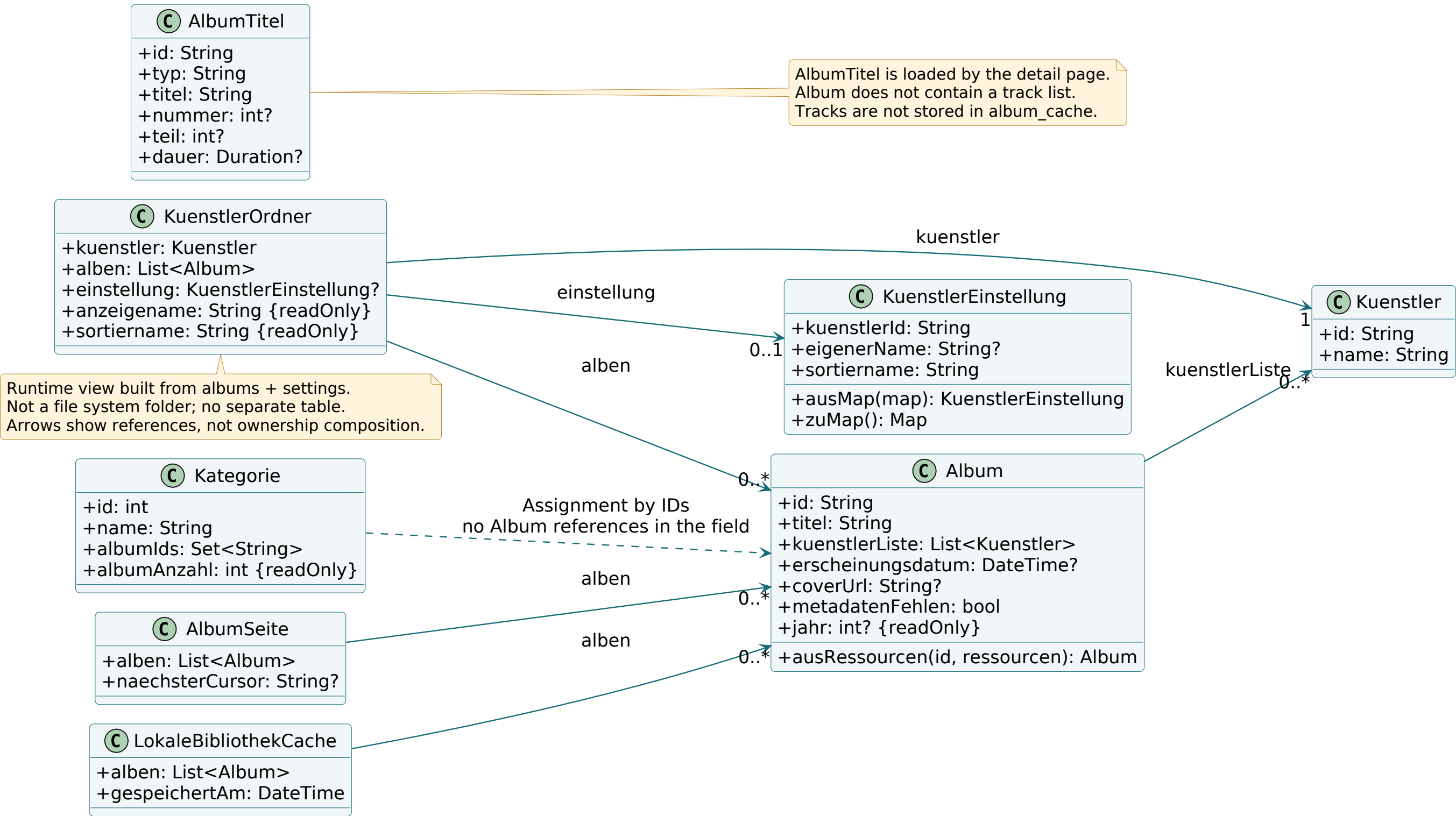
Editable source: PlantUML/01_Komponenten.puml · Vector output: SVG/01_Komponenten.svg

Model 02 · Domain models in memory

Objects, domain relationships and SQL tables are different views. Of particular importance: categories store IDs, and album tracks exist only in the detail page state.

Source locations under lib/, unless stated otherwise: models/modelle.dart; core/sammlung.dart

Domain models in memory

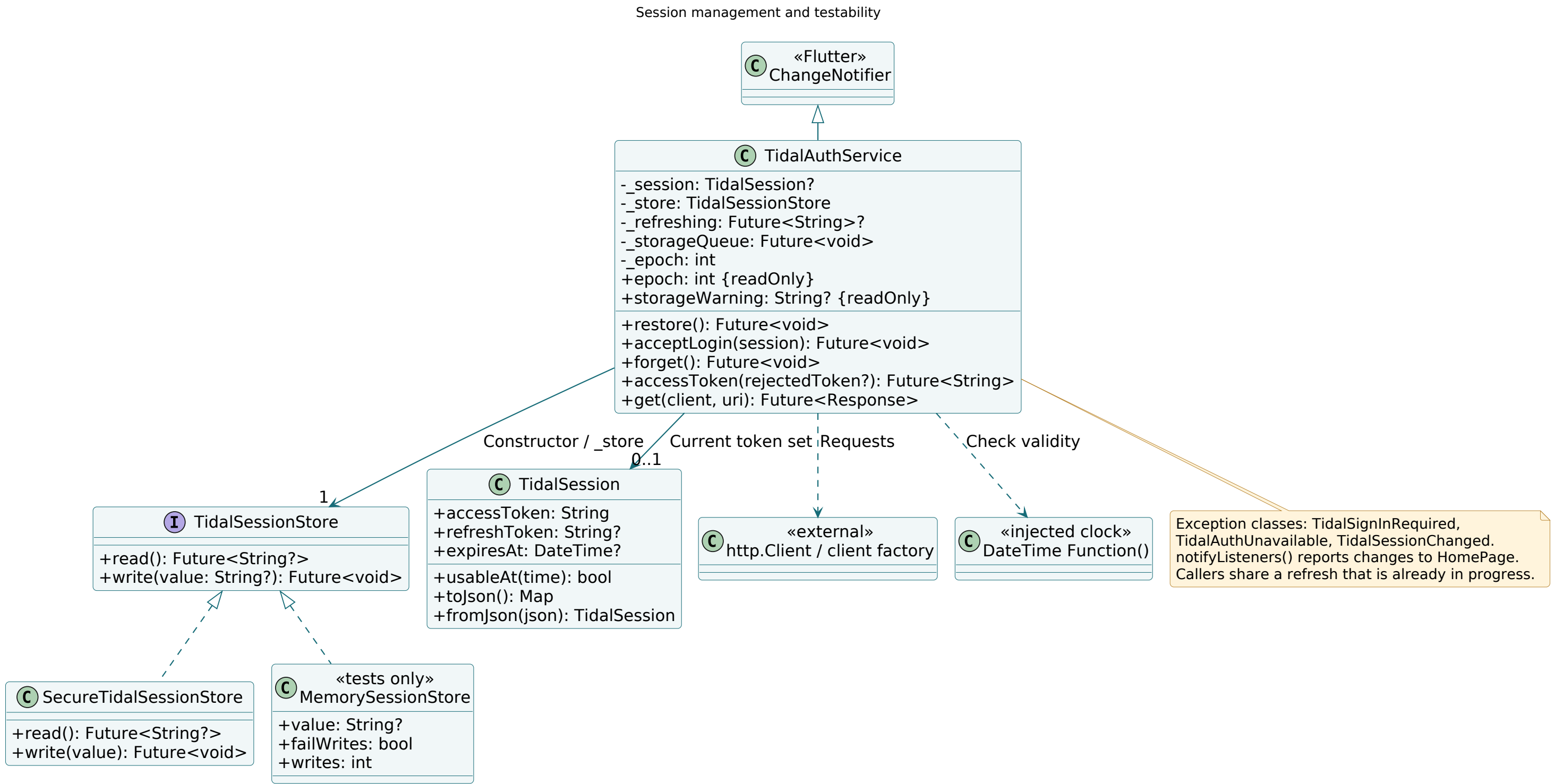


Editable source: PlantUML/02_Klassen_Datenmodelle.puml · Vector output: SVG/02_Klassen_Datenmodelle.svg

Model 03 · Session management and testability

The store, clock and HTTP client are interchangeable. Expiry times, storage failures and concurrent requests can therefore be tested without real TIDAL access.

Source locations under lib/, unless stated otherwise: services/tidal_auth_service.dart; test/support/memory_session_store.dart

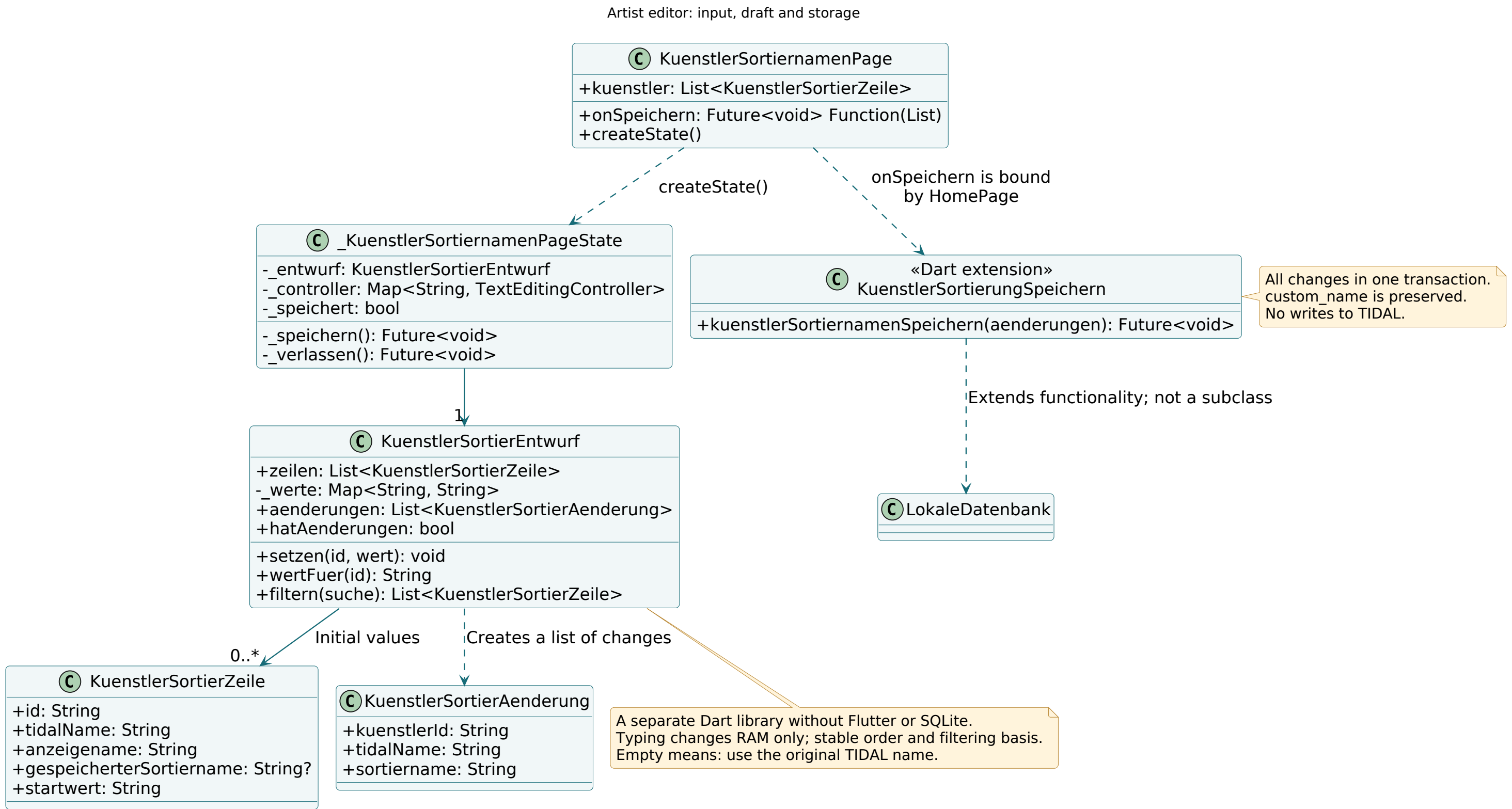


Editable source: PlantUML/03_Klassen_Sitzung.puml · Vector output: SVG/03_Klassen_Sitzung.svg

Model 04 · Artist editor: input, draft and storage

The draft separates provisional input from the stored state. A Dart extension is not inheritance and therefore has no generalisation arrow.

Source locations under lib/, unless stated otherwise: models/kuenstler_sortierung.dart; pages/kuenstler_sortiernamen_page.dart; data/kuenstler_sortierung_speichern.dart

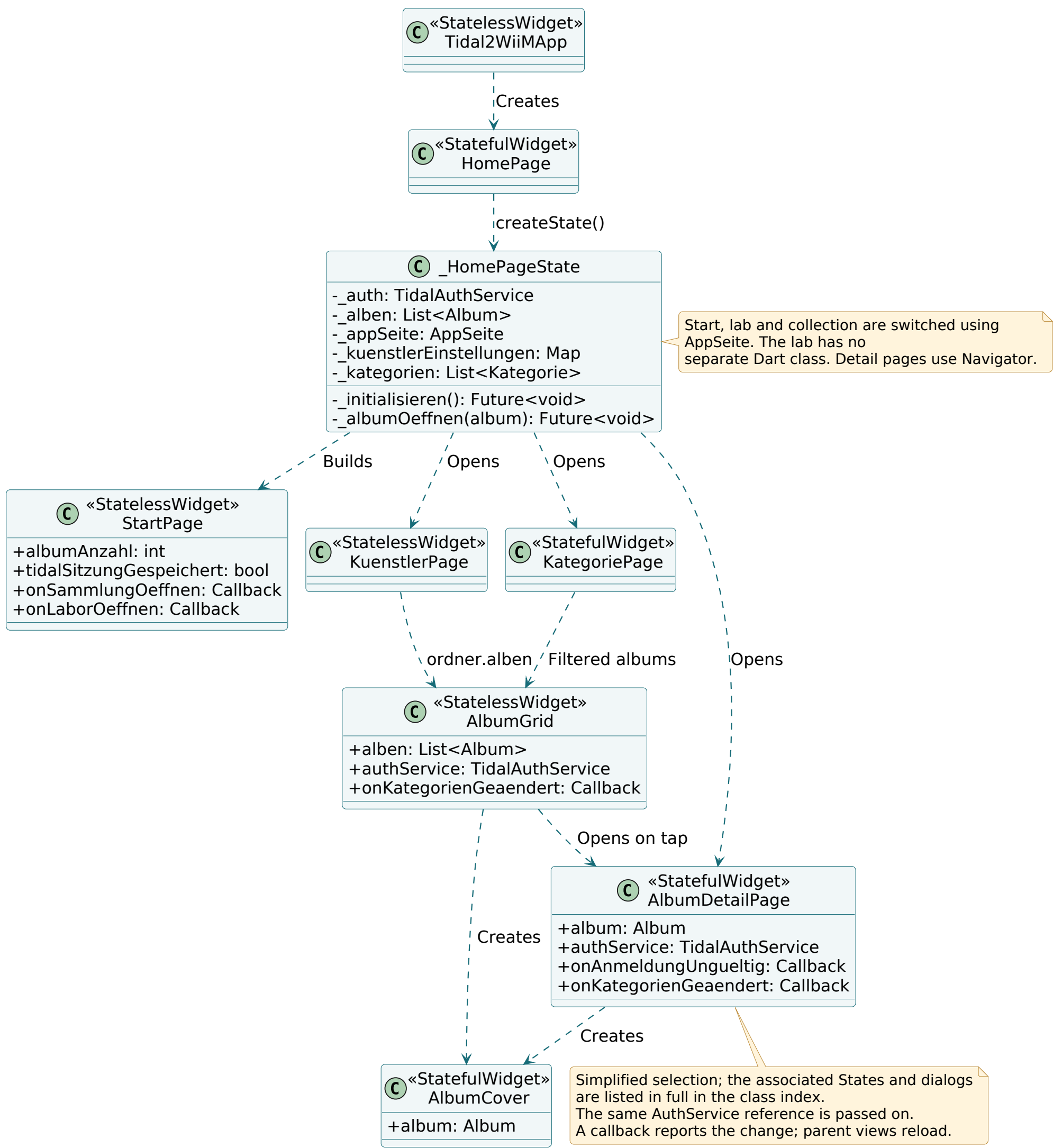


Model 05 · User interface: objects and callbacks

Widgets receive data and functions through constructors. A callback can trigger a reload of the parent view after saving.

Source locations under lib/, unless stated otherwise: main.dart; pages/*; widgets/album_grid.dart; widgets/album_cover.dart

User interface: objects and callbacks

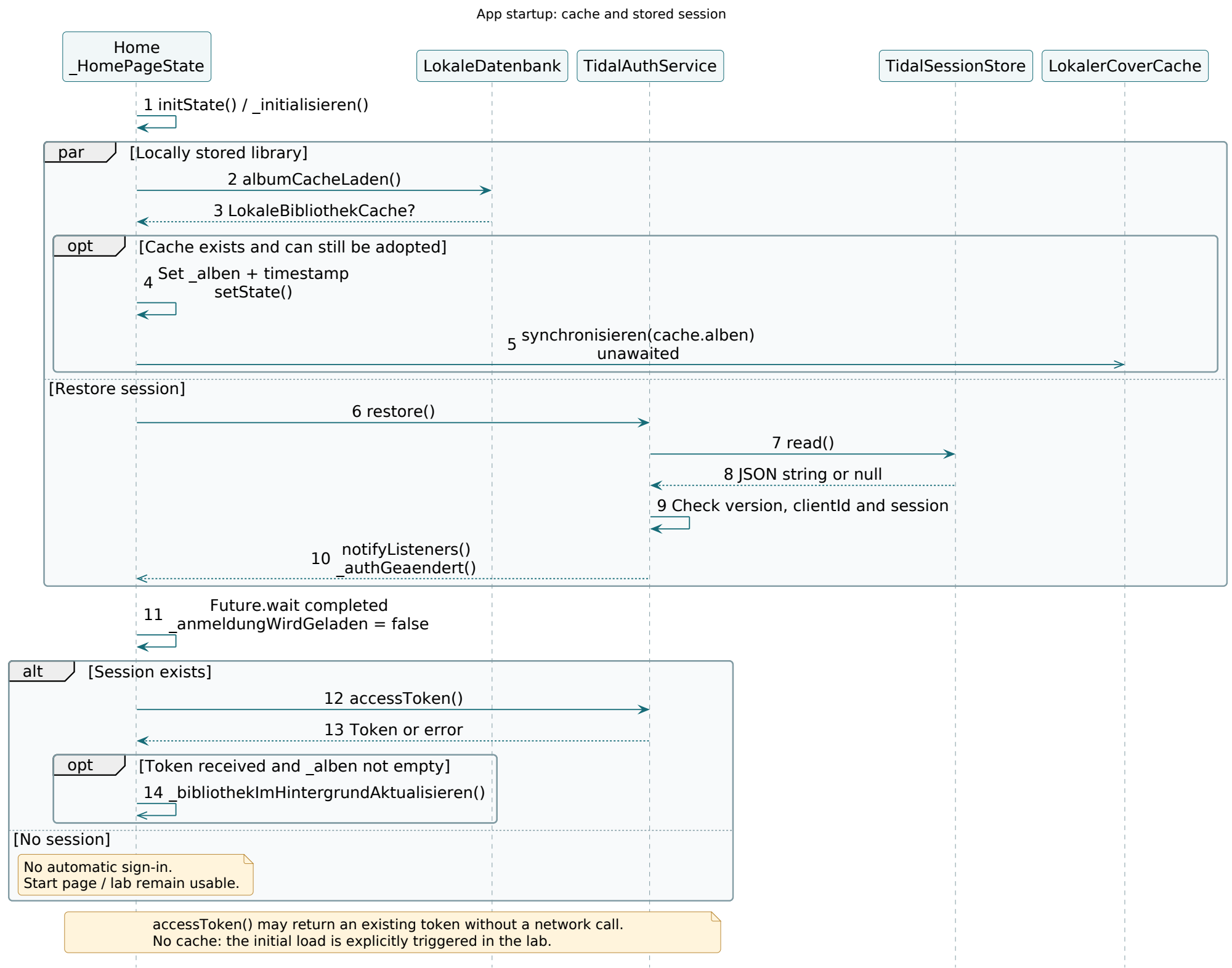


Editable source: PlantUML/05_Klassen_Oberflaeche.puml · Vector output: SVG/05_Klassen_Oberflaeche.svg

Model 06 · App startup: cache and stored session

Two asynchronous tasks start together. A loaded cache is displayed independently of the completion of a later network update; tokens and albums are read separately.

Source locations under lib/, unless stated otherwise: pages/home_page.dart: _initialisieren(), _bibliothekAusCacheLaden(); services/tidal_auth_service.dart: restore()

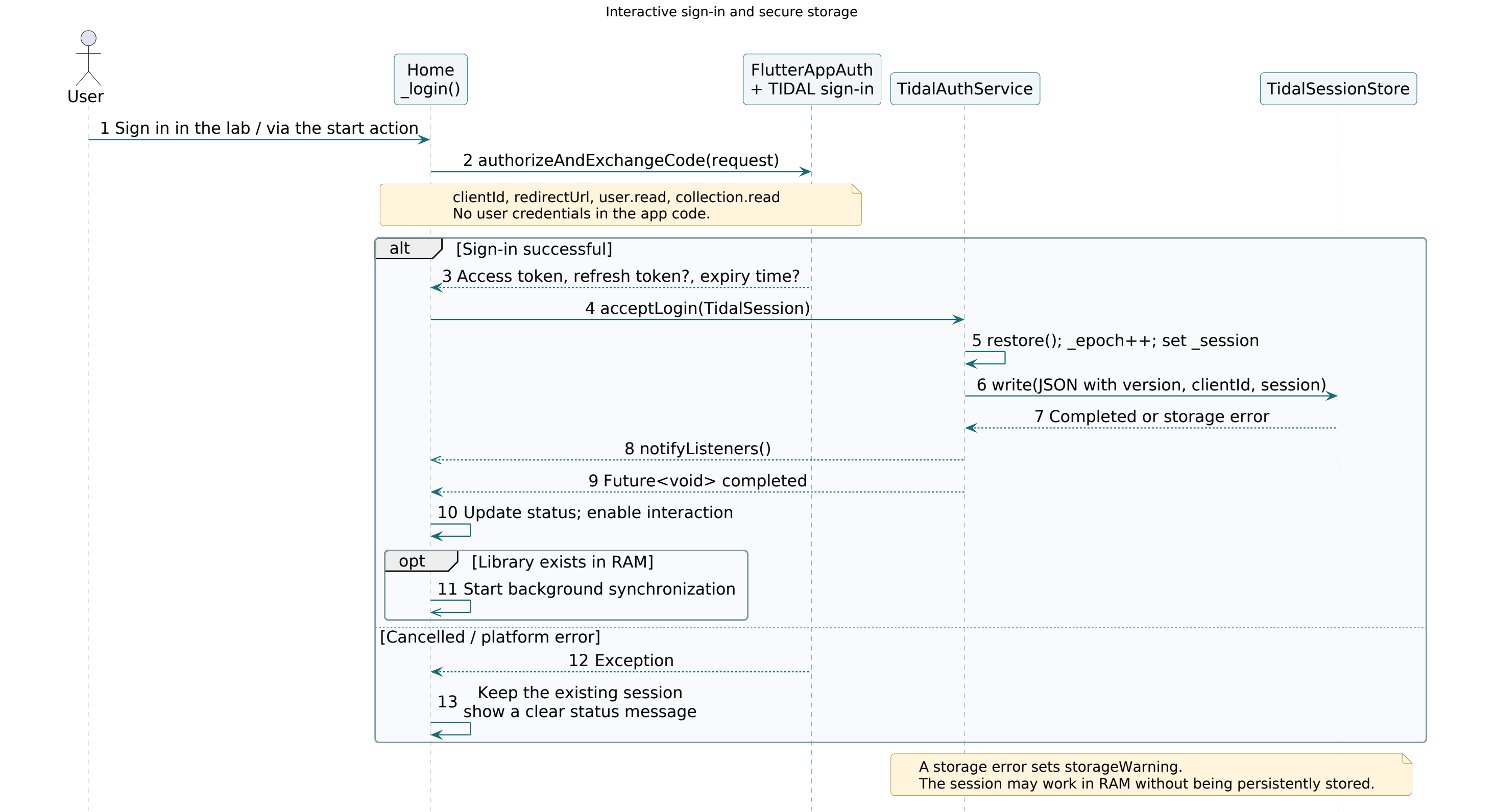


Editable source: PlantUML/06_Sequenz_Appstart.puml · Vector output: SVG/06_Sequenz_Appstart.svg

Model 07 · Interactive sign-in and secure storage

Successful sign-in and successful persistent storage are two different results. This distinction prevents false promises about the next app startup.

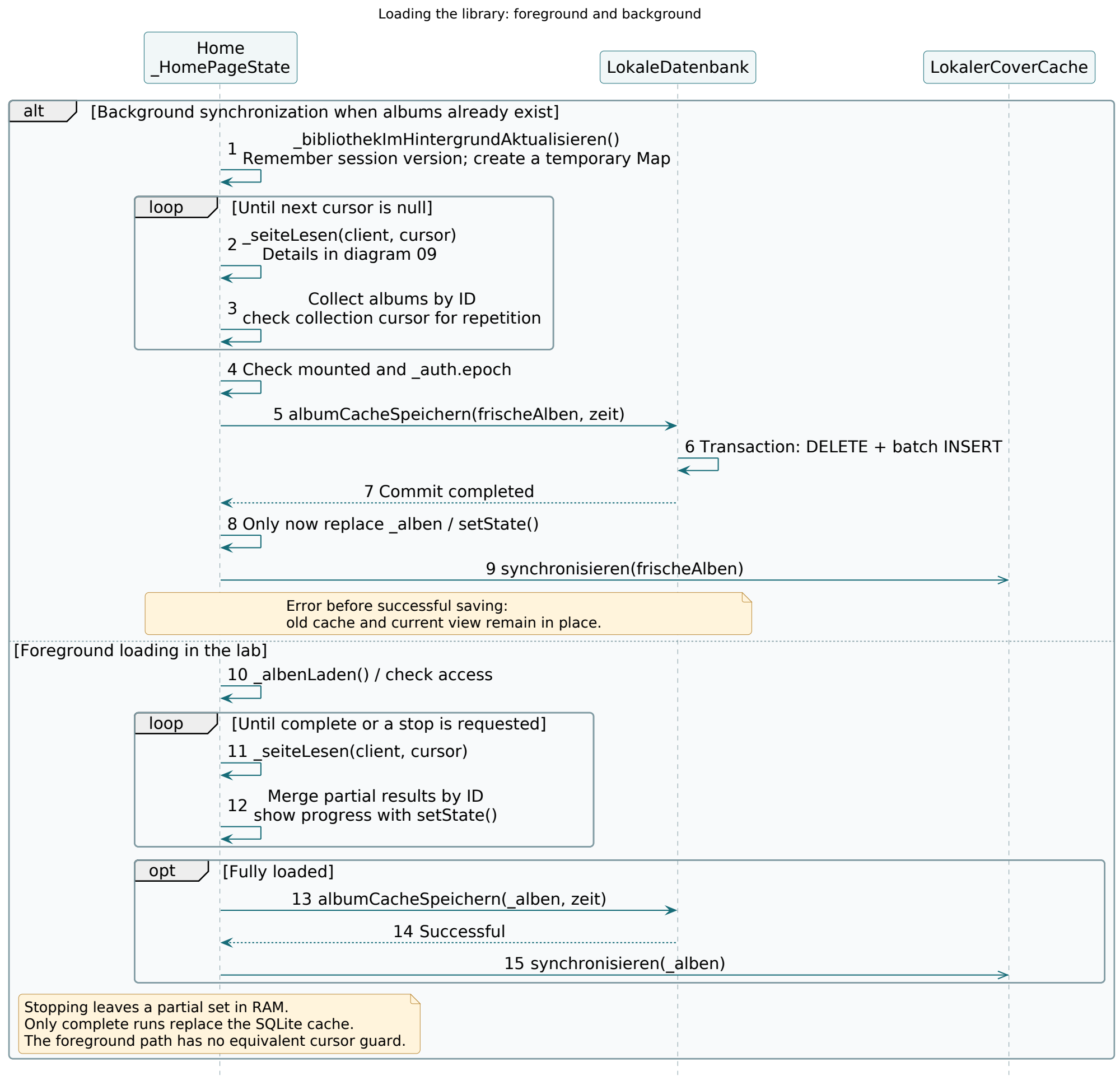
Source locations under lib/, unless stated otherwise: pages/home_page.dart: _login(); services/tidal_auth_service.dart: acceptLogin(), _persist()



Model 08 · Loading the library: foreground and background

During background synchronisation, the old view is retained until the complete new state is available. Foreground loading, by contrast, displays subsets that have already been read.

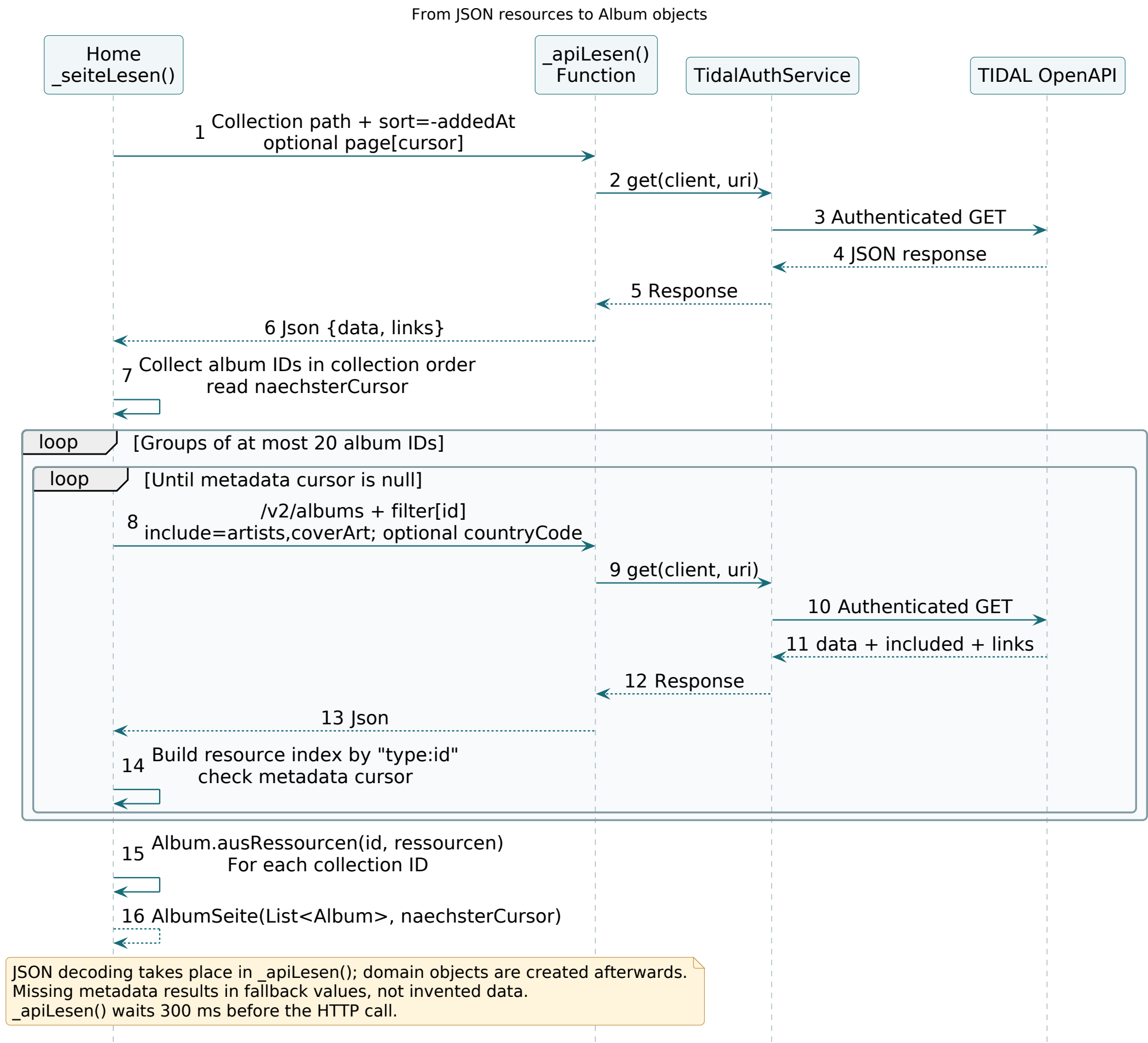
Source locations under lib/, unless stated otherwise: pages/home_page.dart: `_albenLaden()`, `_bibliothekImHintergrundAktualisieren()`; data/lokale_datenbank.dart: `albumCacheSpeichern()`



Model 09 · From JSON resources to Album objects

Collection IDs and album metadata come from separate requests. The resource index resolves the relationships to artists and covers.

Source locations under lib/, unless stated otherwise: pages/home_page.dart: _seiteLesen(); services/tidal_api.dart; models/modelle.dart: Album.ausRessourcen()

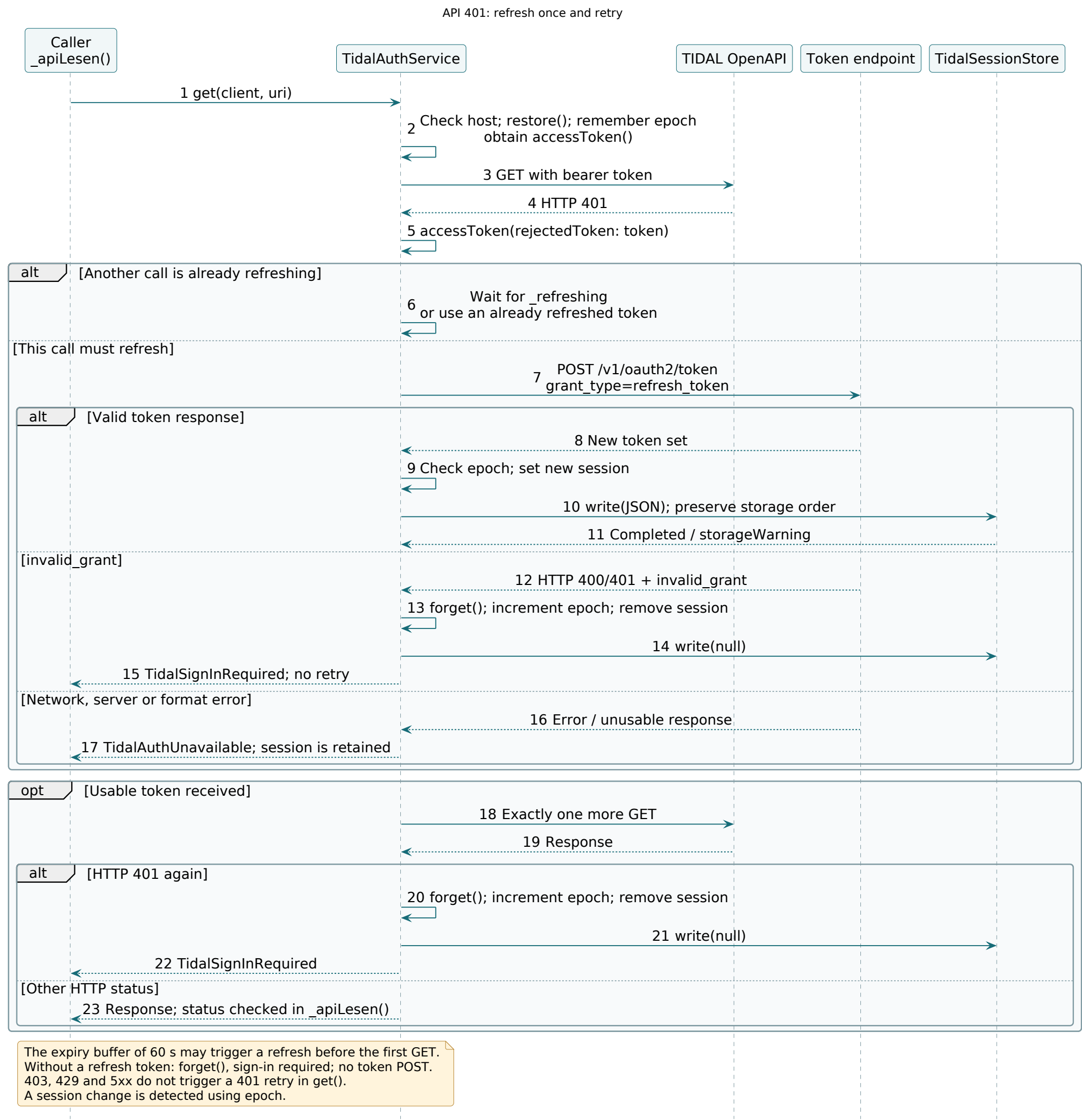


Editable source: PlantUML/09_Sequenz_Albumseite.puml · Vector output: SVG/09_Sequenz_Albumseite.svg

Model 10 · API 401: refresh once and retry

A network error is not evidence of an invalid sign-in. Concurrent calls share the refresh; a second rejection ends without an infinite loop.

Source locations under lib/, unless stated otherwise: services/tidal_auth_service.dart: get(), accessToken(), _refresh(), _persist()

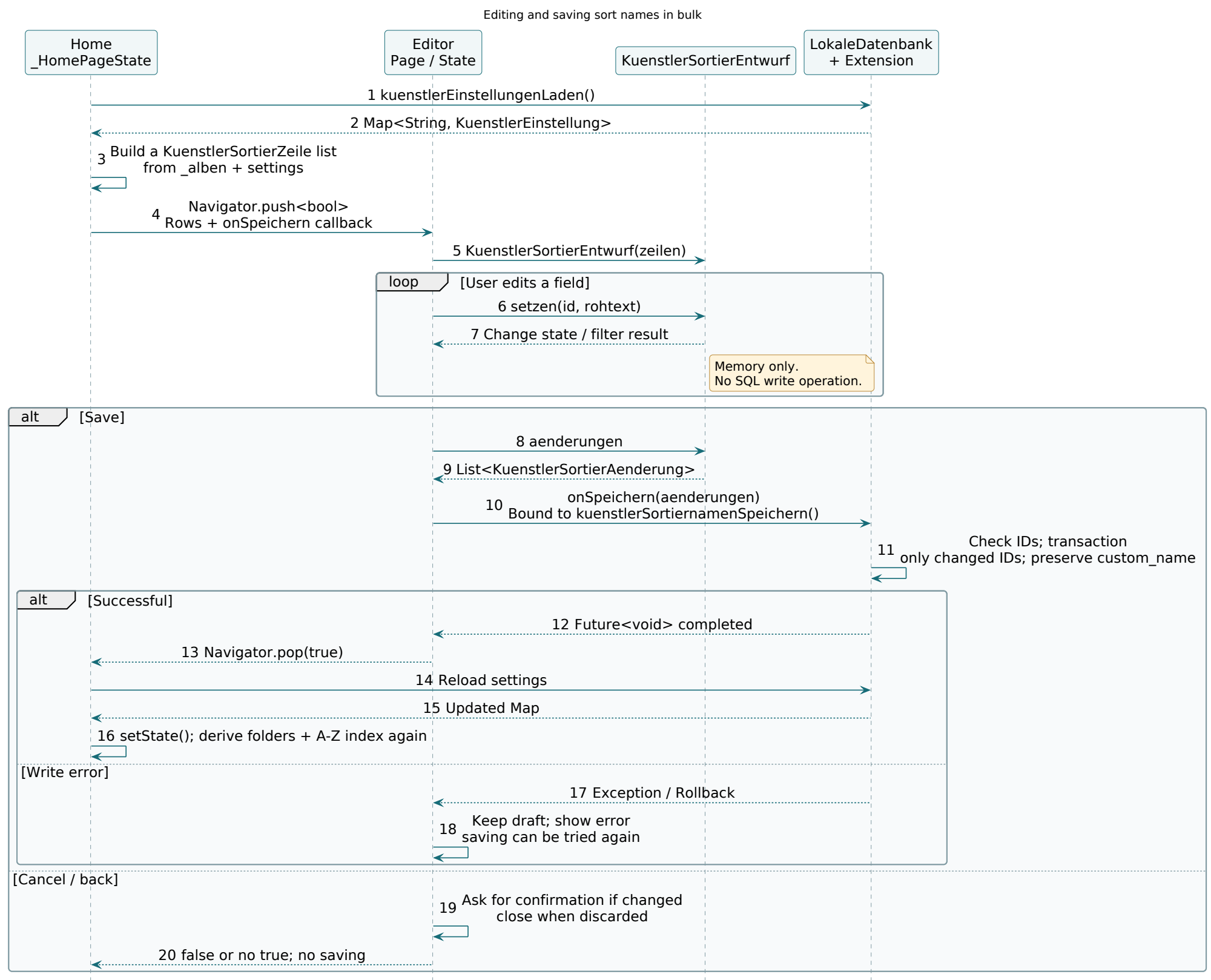


Editable source: PlantUML/10_Sequenz_Token401.puml · Vector output: SVG/10_Sequenz_Token401.svg

Model 11 · Editing and saving sort names in bulk

No saving on every keystroke: a confirmed change list is written together. The page knows only its save callback, not the specific SQL.

Source locations under lib/, unless stated otherwise: pages/home_page.dart: _kuenstlerSortiernamenOeffnen(); pages/kuenstler_sortiernamen_page.dart; models/kuenstler_sortierung.dart; data/kuenstler_sortierung_speichern.dart

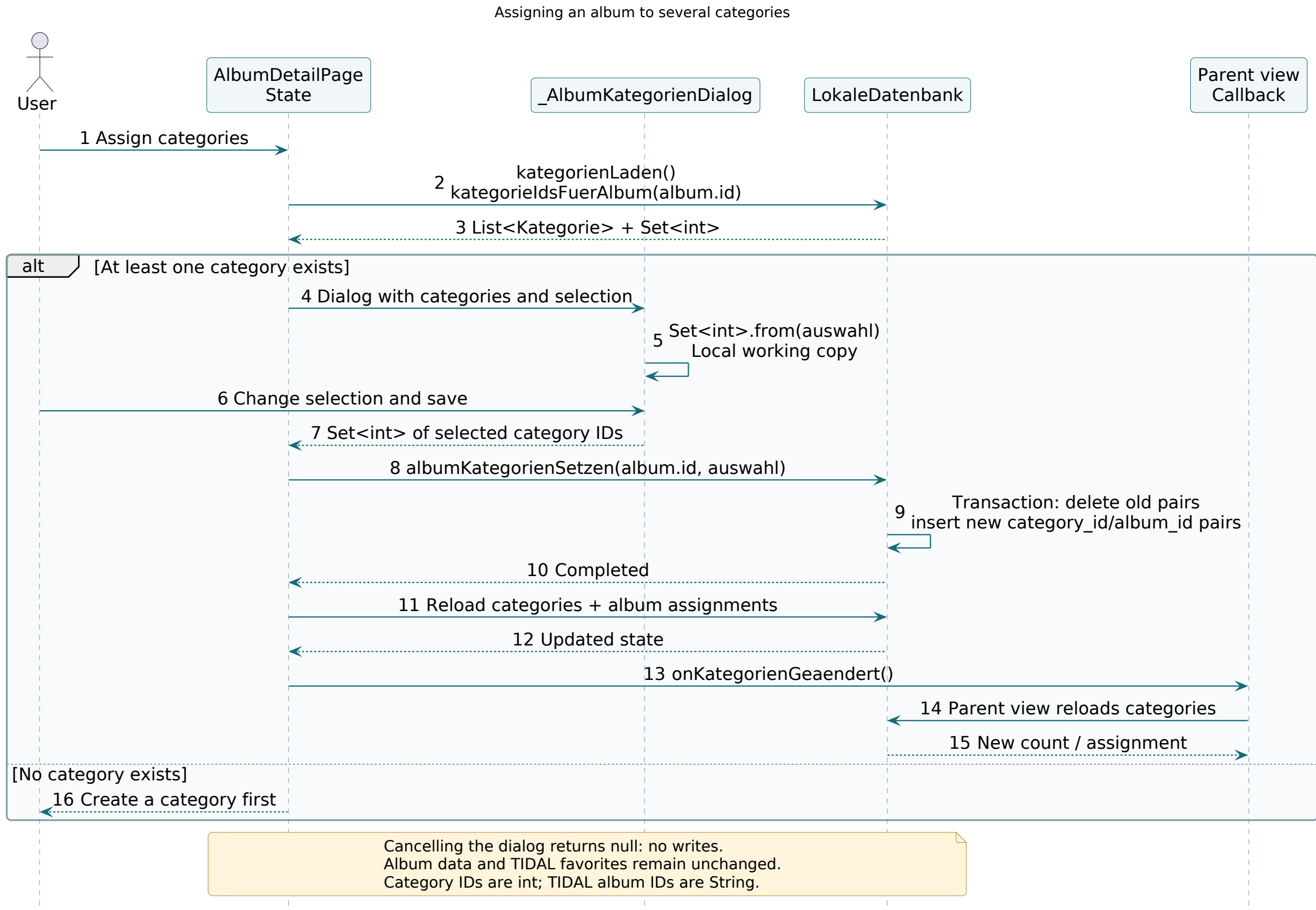


Editable source: PlantUML/11_Sequenz_Kuenstlereditor.puml · Vector output: SVG/11_Sequenz_Kuenstlereditor.svg

Model 12 · Assigning an album to several categories

The checkboxes return only IDs. The detail page then initiates the transaction and reports the change to parent pages.

Source locations under lib/, unless stated otherwise: pages/album_detail_page.dart: _kategorienBearbeiten(); widgets/dialogue.dart: albumKategorienSetzen()

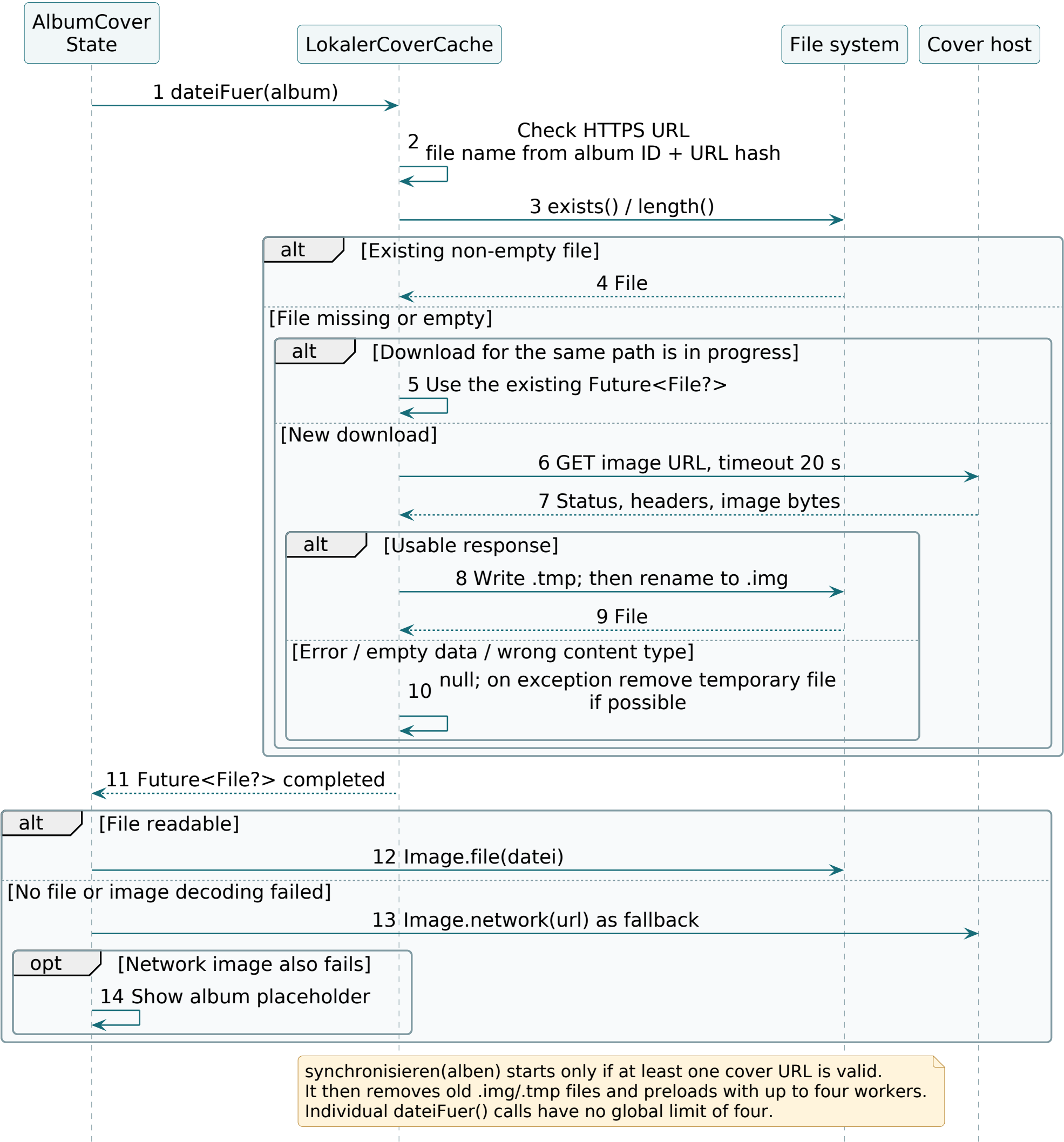


Model 13 · Cover display: file, download and fallback

A File is not an album record. The UI receives a file or null; cover requests do not use an OAuth Bearer header.

Source locations under lib/, unless stated otherwise: services/cover_cache.dart; widgets/album_cover.dart

Cover display: file, download and fallback

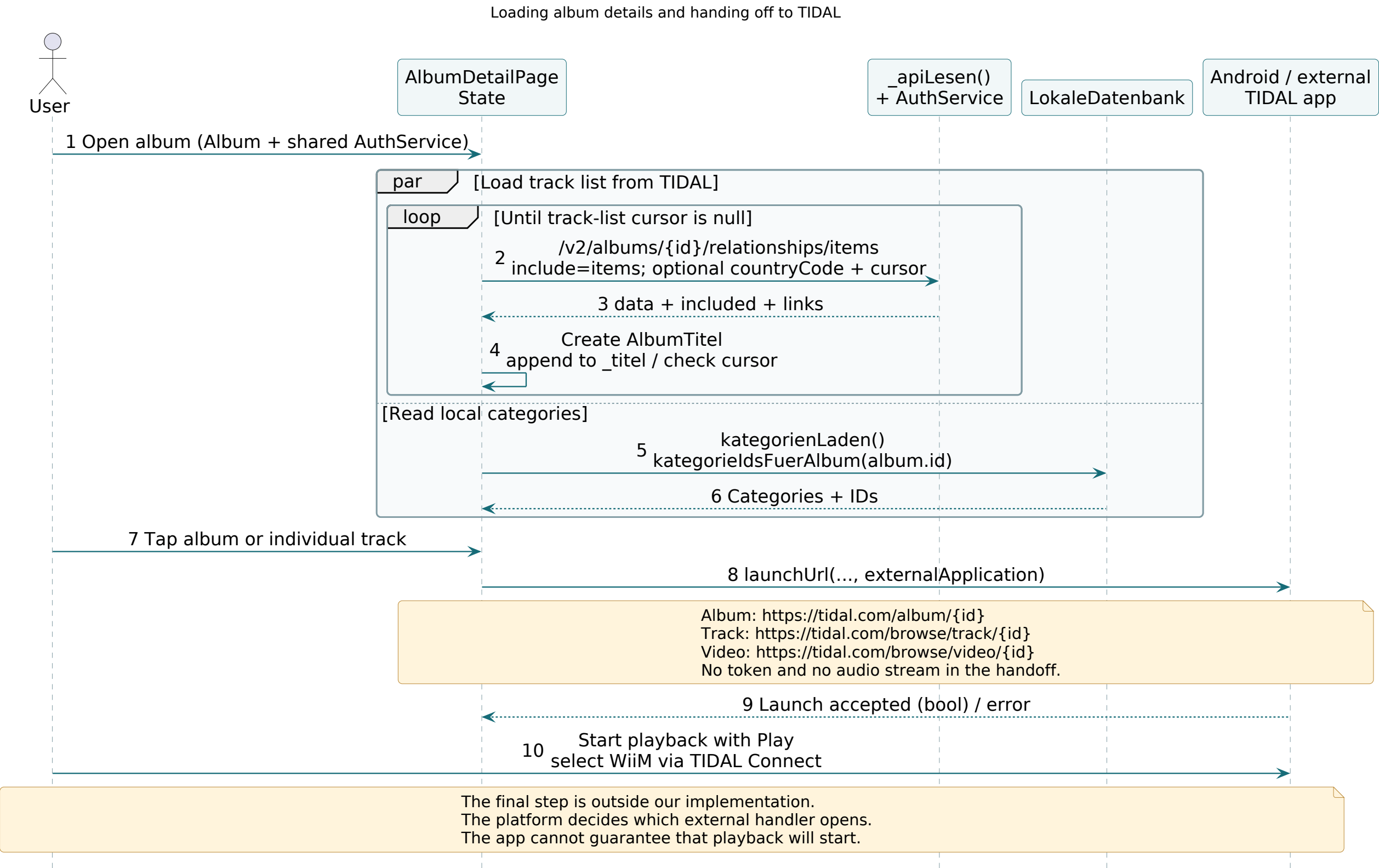


Editable source: PlantUML/13_Sequenz_Cover.puml · Vector output: SVG/13_Sequenz_Cover.svg

Model 14 · Loading album details and handing off to TIDAL

The track list and categories come from different sources. An external link hands off to another app and is not a music player implemented here.

Source locations under lib/, unless stated otherwise: pages/album_detail_page.dart: initState(), _titelLaden(), _inTidalOeffnen(), _titelInTidalOeffnen()



External packages and tests

Direct external packages

The listed package versions come from the pubspec.lock of the documented source revision.

Package	Lockfile version	Responsibility
flutter	SDK	Widgets, MaterialApp, Navigator, setState, ChangeNotifier and lifecycle.
flutter_appauth	12.1.0	Interactive TIDAL sign-in; supplies the initial token set to HomePage.
http	1.6.0	OpenAPI GET, token POST and cover downloads; interchangeable clients enable testing.
sqlite	2.4.4	SQLite queries, migrations and transactions on the device.
path	1.9.1	Joins database and cover paths appropriately for the platform.
flutter_secure_storage	11.2.0	Secure platform storage for the JSON token package.
url_launcher	6.3.2	Passes album, track and video links to an external application.
cupertino_icons	1.0.9	Declared in the project; no CupertinoIcons call in the examined lib code.
flutter_test	SDK	Unit/widget tests, WidgetTester and the Flutter test environment.
flutter_lints	6.0.0	Static style/analysis rules through analysis_options.yaml; not a runtime service.
flutter_launcher_icons	0.14.4	Development tool for generating app icons; not a runtime service.

The Dart SDK libraries dart:async, dart:convert and dart:io provide asynchronous operations, JSON/UTF-8 and file access. Flutter provides material.dart, services.dart and foundation.dart, among others. These imports are not additional project-specific packages.

Configuration and platform folders

pubspec.yaml describes the package, dependencies and assets. pubspec.lock records the resolved versions. analysis_options.yaml configures static analysis. assets/ contains the start image and icon. docs/datenmodell/ contains the existing planning models.

android/ contains the native Android embedding, Gradle configuration, manifest, OAuth callback and resources. ios/, macos/, linux/ and web/ are additional platform scaffolds. Their existence does not demonstrate tested support for these platforms. The declared target device remains Android.

Existing tests

File	Focus	What they check / provide
test/kuenstler_sortierung_test.dart	Draft logic	Only changed IDs, fallback for empty values, stable filters/order, separate IDs despite identical names and invalid input.
test/kuenstler_sortiernamen_page_test.dart	Editor UI	Save/cancel, Android back navigation, save errors, preserved input, keyboard focus and different screen/text sizes.
test/tidal_auth_service_test.dart	Session service	Restoration, expiry time, shared refresh, network/server errors, invalid_grant, storage warning and exactly one 401 retry.
test/widget_test.dart	Start/laboratory	Start actions and touch targets, dynamic status values, restoration/forgetting sign-in and layout variants.
test/support/memory_session_store.dart	Test double	MemorySessionStore stores only in RAM, counts writes and simulates write errors; not an independent test case.

The overview describes the existing test sources. Results from rerunning tests or builds are not part of this documentation. Widget tests with injected data or mock clients check isolated processes; real API, SQLite or tablet integration requires additional integration tests.

There is no dedicated direct integration test for SQLite transactions or the cover cache in the examined test/ folder. Complete end-to-end verification with TIDAL and WiiM would be an additional check.

Complete class index

46 class and interface declarations in lib/. The module identifier points to the profile with the full file path. Framework classes and MemorySessionStore from the tests are not included in this count.

Name	Module	Responsibility
LokaleDatenbank	M06	SQLite access and schema 3.
Tidal2WiiMApp	M01	Root widget, theme and HomePage.
KuenstlerSortierZeile	M05	Unchanged initial data for an editor row.
KuenstlerSortierAenderung	M05	Data object for a confirmed sort-name change.
KuenstlerSortierEntwurf	M05	In-memory state for input, filtering and change detection.
KuenstlerEinstellung	M04	Custom display name and local sort name.
Kategorie	M04	Category ID, name and set of album IDs.
Kuenstler	M04	TIDAL artist ID and name.
Album	M04	Album metadata and participating artists.
LokaleBibliothekCache	M04	Loaded album list with storage timestamp.
AlbumTitel	M04	A track or video in the detail page state.
KuenstlerOrdner	M04	Artist view derived from albums.
AlbumSeite	M04	Result of an API collection page with the next cursor.
AlbumDetailPage	M13	Configuration for a selected album.
_AlbumDetailPageState	M13	Track list, categories and handoff to an external application.
HomePage	M11	Main page configuration and optional test dependencies.
_HomePageState	M11	Central coordination of state, data loading and navigation.
KategoriePage	M15	Configuration of a category view.
_KategoriePageState	M15	Reads the category, filters current album objects and refreshes the display.
KuenstlerPage	M14	Artist heading and shared album grid.
KuenstlerSortiernamenPage	M16	Editor configuration and save callback.
_KuenstlerSortiernamenPageState	M16	Draft, input fields, focus and cancel/save flow.
StartPage	M12	Start image and status data supplied by the parent widget.
_StartAktion	M12	Labelled, usable touch target.
_StartStatusLeiste	M12	Album/TIDAL status and refresh action.
_StartTidalZeichen	M12	Draws the status symbol as a CustomPainter.

Name	Module	Responsibility
_StartBibliothekZeichen	M12	Draws the library symbol as a CustomPainter.
LokalerCoverCache	M10	File lookup, download, preloading and cover cleanup.
ApiFehler	M09	HTTP status and API path as an error object.
TidalSessionStore	M08	Interface for reading/writing the session package.
SecureTidalSessionStore	M08	Production store based on flutter_secure_storage.
TidalSession	M08	Token set and expiry time.
TidalSignInRequired	M08	A new sign-in is required.
TidalAuthUnavailable	M08	Temporary access failure without automatically discarding the session.
TidalSessionChanged	M08	An operation in progress no longer belongs to the current session.
TidalAuthService	M08	Shared session, token refresh and authenticated API GETs.
AlbumCover	M18	Configuration of the cover display for an album.
_AlbumCoverState	M18	File Future, loading indicator and image fallback.
AlbumGrid	M17	Shared grid for artist/category child pages.
_KategorieNameDialog	M19	Configuration of the category name dialog.
_KategorieNameDialogState	M19	Text controller, required-field validation and name return.
_KuenstlerDialogErgebnis	M19	Setting or explicit request to reset.
_KuenstlerBearbeitenDialog	M19	Configuration of an individual artist adjustment.
_KuenstlerBearbeitenDialogState	M19	Controllers for display/sort names and result return.
_AlbumKategorienDialog	M19	Configuration of the category selection.
_AlbumKategorienDialogState	M19	Edits a copy of the selected IDs.

Also present: the Dart extension KuenstlerSortierungSpeichern (M07), the enums Sortierung, SammlungAnsicht and AppSeite (M03), and the Json type alias (M02). The functions in core/ and tidal_api.dart are not additional classes.

Working with the sources

The ZIP archive is a standalone documentation package. It contains the reading version, Markdown chapters, 19 individually usable module profiles and the 14 models as PlantUML, SVG and PNG files. The app sources are not included a second time; the profiles link to the fixed GitHub revision.

Rendering the models

Java and an existing plantuml.jar are sufficient. The supplied scripts write their output to SVG/ and PNG/.

```
.\Modelle_rendern.ps1 -PlantUmlJar "C:\Tools\plantuml\plantuml.jar"
```

Or with Python and Java:

```
python Modelle_rendern.py /path/to/plantuml.jar
```

This version was rendered locally with PlantUML 1.2025.10. Component and class models use Smetana; no external Graphviz installation is required for them. No project code was sent to a public rendering service.

Documenting a change so it can be traced

1. Read the relevant module profile and its associated model.
2. First clarify which data changes and where it is stored.
3. Update the code and PlantUML source together, then render again.
4. Support statements with appropriate checks and state the source revision.
5. Explicitly state the tools used, including ChatGPT.